

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №6
по дисциплине «Вычислительная математика»
Тема: Метод простых итераций

Студент гр. 4342

Митюков М.Н.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Изучить метод простых итераций для численного решения уравнения $f(x)=0$, получить приближённое значение корня с заданной точностью, а также исследовать скорость сходимости и обусловленность метода.

Задание.

Метод простых итераций решения уравнения $f(x) = 0$ состоит в замене исходного уравнения эквивалентным ему уравнением $x = \varphi(x)$ и построении последовательности $x_{n+1} = \varphi(x_n)$, сходящейся при $n \rightarrow \infty$ к точному решению. Достаточные условия сходимости метода простых итераций формулируются теоремой, приведенной [1,2,7].

Рассмотрим один шаг итерационного процесса. Исходя из найденного на предыдущем шаге значения x_{n-1} , вычисляется $y = \varphi(x_{n-1})$. Если $|y - x_{n-1}| > \varepsilon$, то полагается $x_n = y$ и выполняется очередная итерация. Если же $|y - x_{n-1}| < \varepsilon$, то вычисления заканчиваются и за приближенное значение корня принимается величина $x_n = y$. Погрешность результата вычислений зависит от знака производной $\varphi'(x)$: при $\varphi'(x) > 0$ погрешность определения корня составляет $q\varepsilon/1-q$, а при $\varphi'(x) < 0$ погрешность не превышает ε . Здесь q - число, такое, что $|\varphi'(x)| \leq q < 1$ на отрезке $[a,b]$. Существование числа q является условием сходимости метода в соответствии с отмеченной выше теоремой.

Для применения метода простых итераций определяющее значение имеет выбор функции $\varphi(x)$ в уравнении $x = \varphi(x)$, эквивалентном исходному. Функцию $\varphi(x)$ необходимо подбирать так, чтобы $|\varphi'(x)| \leq q < 1$. Это обуславливается тем, что если $\varphi'(x) > 0$ на отрезке $[a,b]$, то последовательные приближения $x_{n+1} = \varphi(x_n)$ будут колебаться около корня α , если же $\varphi'(x) < 0$, то последовательные приближения будут сходиться к корню α монотонно. Следует также помнить, что скорость сходимости последовательности $\{x_n\}$ к корню α функции $f(x)$ тем выше, чем меньше число q .

В лабораторной работе № 6 предлагается, используя программы функции ITER и Round из файла methods.cpp (файл заголовков methods.h, директория LIBR11), найти корень уравнения $f(x) = 0$ с заданной

точностью ϵ_{rs} методом итераций, исследовать скорость сходимости и обусловленность метода.

Для данной работы вид функции $f(x)$ задается индивидуально каждому студенту преподавателем из числа вариантов, приведенных в подразделе 3.6.

Порядок выполнения лабораторной работы №6 должен быть следующим:

- 1) Графически или аналитически отделить корень уравнения $f(x) = 0$.
- 2) Преобразовать уравнение $f(x) = 0$ к виду $x = \varphi(x)$ так, чтобы в некоторой окрестности $[Left, Right]$ корень производная $\varphi'(x)$ удовлетворяла условию $|\varphi'(x)| \leq 1$. При этом следует иметь в виду, что чем меньше величина q , тем быстрее последовательные приближения сходятся к корню.
- 3) Выбрать начальное приближение, лежащее на $[Left, Right]$.
- 4) Составить подпрограмму для вычисления значений $\varphi(x)$, $\varphi'(x)$, предусмотреть округление вычисленных значений с точностью Δ .
- 5) Составить головную программу, вычисляющую корень уравнения и содержащую обращение к программам $\varphi(x)$, $\varphi'(x)$ и ITER в индикацию результатов.
- 6) Провести вычисления по программе. Исследовать скорость сходимости и обусловленность метода.

Текст программы - функции ITER, позволяющей вычислить корень уравнения $x = \varphi(x)$ для любой функции, которая удовлетворяет достаточным условиям сходимости метода, приводится ниже.

Выполнение работы.

1. Отделение (изоляция) корня уравнения.

Функция $f(x) = \sin(x^2) - 6x + 1$ непрерывна и дифференцируема на $\mathbb{R}$.

Производная исходной функции имеет вид

$$f'(x) = 2x \cdot \cos(x^2) - 6.$$

1. При $x = 0$:

$$f(0) = \sin(0) - 6 \cdot 0 + 1 = 1 > 0.$$

2. При $x = 0.5$:

$$f(0.5) = \sin(0.5^2) - 6 \cdot 0.5 + 1 = \sin(0.25) - 3 \approx 0.247 - 3 \approx -2.753 < 0.$$

Так как $f(0) > 0$ и $f(0.5) < 0$, то: $f(0) \cdot f(0.5) < 0$.

Так как $f(0) \cdot f(0.5) < 0$, то по теореме Больцано на интервале $(0; 0.5)$ существует корень уравнения $f(x) = 0$. На выбранном отрезке производная отрицательна, следовательно, функция строго убывает, корень единственный.

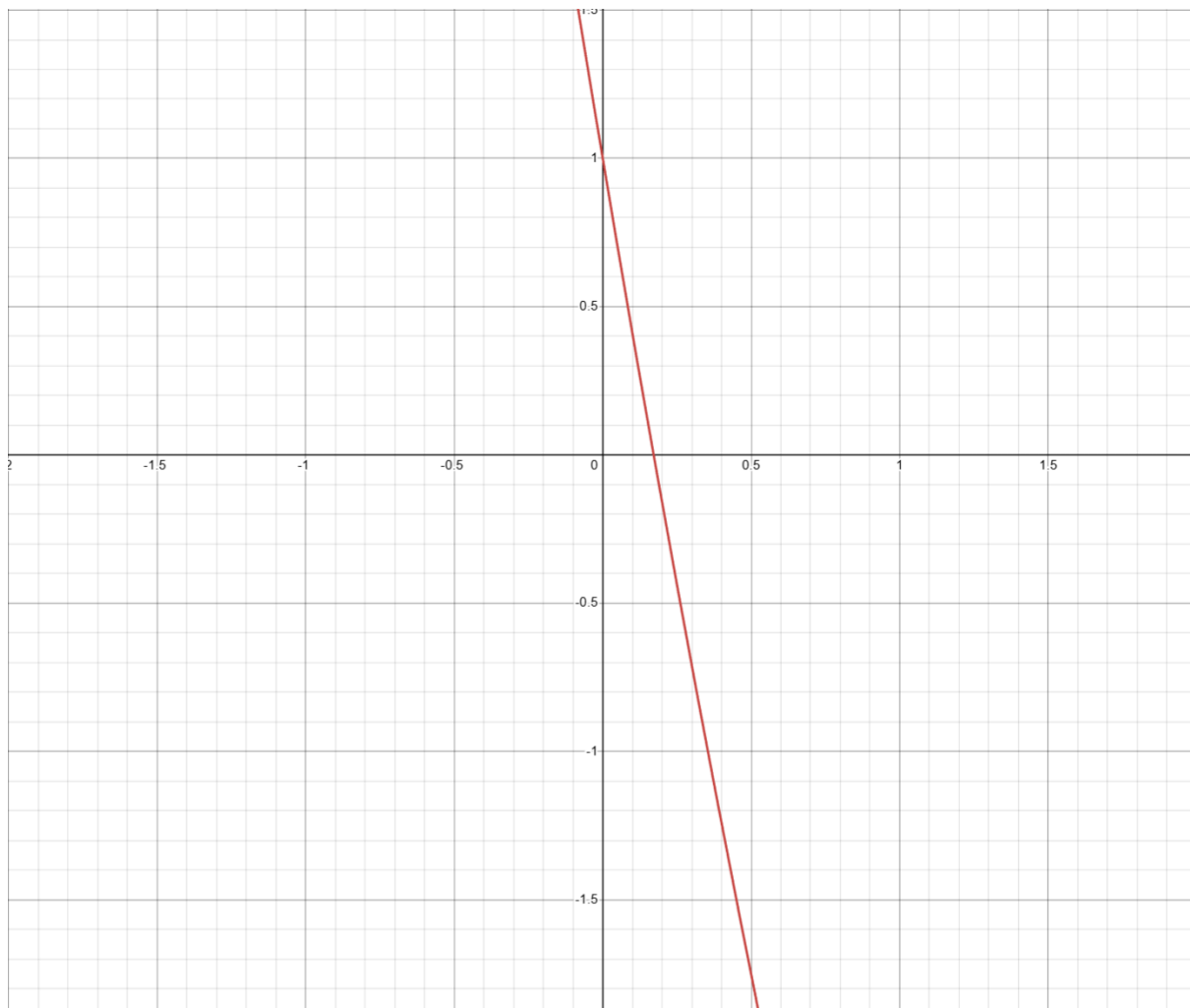


Рисунок 1 — График функции $f(x) = \sin(x^2) - 6x + 1$ и отрезок изоляции корня $[0; 0.5]$.

2. Преобразование уравнения к виду $x = \phi(x)$.

Для применения метода простых итераций исходное уравнение перепишем в форме

$$x = \phi(x), \quad \phi(x) = (\sin(x^2) + 1)/6.$$

$$\phi'(x) = \frac{x \cdot \cos(x^2)}{3}$$

$$\phi([0; 0.5]) \in [0.333333; 0.415801]$$

$$q = \max(|\phi'(x)|) = 0.161485$$

На отрезке $[0; 0.5]$ выполняется оценка $|\phi'(x)| \leq q \approx 0.161485 < 1$. Следовательно, достаточное условие сходимости метода простых итераций выполнено.

Так как производная $\varphi'(x)$ на выбранном интервале положительно, приближение к корню будет монотонно.

В качестве рабочего интервала итераций принимаем $[0; 0.5]$.

В качестве начального приближения удобно взять середину этого отрезка.

$$x_0 = (0 + 0.5) / 2 = 0.25.$$

3. Подпрограммы вычисления $f(x)$, $\varphi(x)$ и $\varphi'(x)$.

В главной программе реализованы три функции: исходная функция $f(x) = \sin(x^2) - 6x + 1$, её производная $f'(x)$ и итерационная функция $\varphi(x) = (\sin(x^2) + 1) / 6$. Значение $\varphi'(x)$ вычисляется по формуле $\varphi'(x) = (x * \cos(x^2)) / 3$.

Для исследования чувствительности метода к ошибкам вычислений используется подпрограмма $\text{Round}(\text{value}, \Delta)$, округляющая значение до ближайшего числа, кратного Δ . При $\Delta = 0$ округление не выполняется.

В экспериментах по обусловленности округление применяется к значению итерационной функции $\varphi(x)$. Это позволяет проследить, как конечная точность промежуточных вычислений влияет на число шагов и на итоговую ошибку найденного корня.

4. Подпрограммы Round , EstimateQ и ITER .

В файле `methods.cpp` реализованы три основные процедуры. Функция Round выполняет округление значения с шагом Δ . Функция EstimateQ оценивает величину $q = \max|\varphi'(x)|$ на отрезке $[\text{Left}; \text{Right}]$ по равномерной сетке. Основная процедура ITER строит последовательность приближений

$$x_{n+1} = \varphi(x_n).$$

На каждом шаге вычисляется новое значение x_{n+1} и разность $|x_{n+1} - x_n|$. Если $|x_{n+1} - x_n| < \text{Eps}$, итерационный процесс прекращается, а значение x_{n+1}

принимается за приближённый корень. Если условие не выполнено, выполняется следующий шаг.

Для рассматриваемой функции метод имеет линейную скорость сходимости, поскольку $q < 1$ на всём рабочем интервале. Практически это означает устойчивое уменьшение ошибки и сравнительно умеренное число итераций.

5. Головная программа и организация экспериментов.

Главная программа задаёт параметры задачи: интервал итераций $[0; 0.5]$, начальное приближение $x_0 = 0.25$ и предельное число итераций $N_{\max} = 1000$. Сначала выполняется опорный расчёт без округления при $Eps = 10^{-14}$, который даёт эталонное значение корня x_{ref} .

Далее проводятся два набора вычислительных экспериментов. Первый набор предназначен для исследования скорости сходимости: точность Eps меняется от 10^{-1} до 10^{-8} , после чего для каждого запуска сохраняются число итераций, найденный корень и абсолютная ошибка $|x - x_{ref}|$.

Второй набор посвящён обусловленности метода. При фиксированной точности $Eps = 10^{-8}$ рассматриваются различные значения шага округления Δ . Для каждого Δ сохраняются число итераций, полученное значение Δx , абсолютная ошибка $|\Delta x - x_{ref}|$ и невязка $|f(\Delta x)|$. Результаты экспериментов записываются в CSV-файлы, а затем по ним строятся графики в программе plot.py.

6. Результаты вычислительных экспериментов.

Опорный расчёт без округления дал значение

$$x_{ref} \approx 0.171572123742972.$$

Для получения этого значения методу простых итераций потребовалось 12 шагов при $Eps = 10^{-14}$. Ниже приведены основные результаты вычислительного эксперимента.

6.1. Исследование скорости сходимости.

В Приложении Б в файле `speed_results.csv` показано, как меняется число итераций при уменьшении требуемой точности Eps . Видно, что метод сходится устойчиво: при ужесточении требования по точности количество шагов возрастает постепенно, что соответствует линейному характеру сходимости.

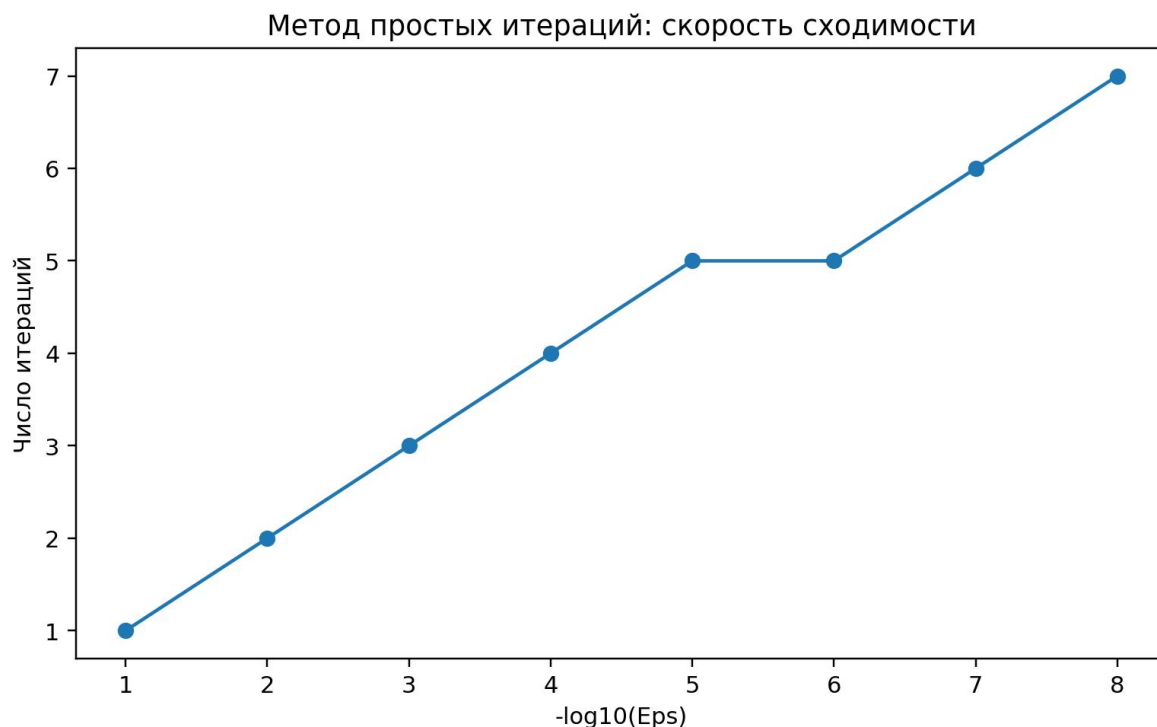


Рисунок 2 — Зависимость числа итераций метода простых итераций от требуемой точности Eps .

По графику видно, что при уменьшении Eps число итераций возрастает почти линейно по отношению к величине $-\log_{10}(\text{Eps})$. Это типичный признак линейной сходимости: на каждом шаге ошибка уменьшается примерно в постоянное число раз, которое определяется величиной q .

6.2. Исследование чувствительности к округлению.

В Приложении Б в файле `conditioning_results.csv` приведены результаты при фиксированной точности $\text{Eps} = 10^{-8}$ и различных значениях

Δ . Абсолютная ошибка вычислялась по формуле $|\Delta x - x_{\text{ref}}|$. Эксперимент показывает, что грубое округление существенно ухудшает результат, а при достаточно малых Δ метод снова работает устойчиво.

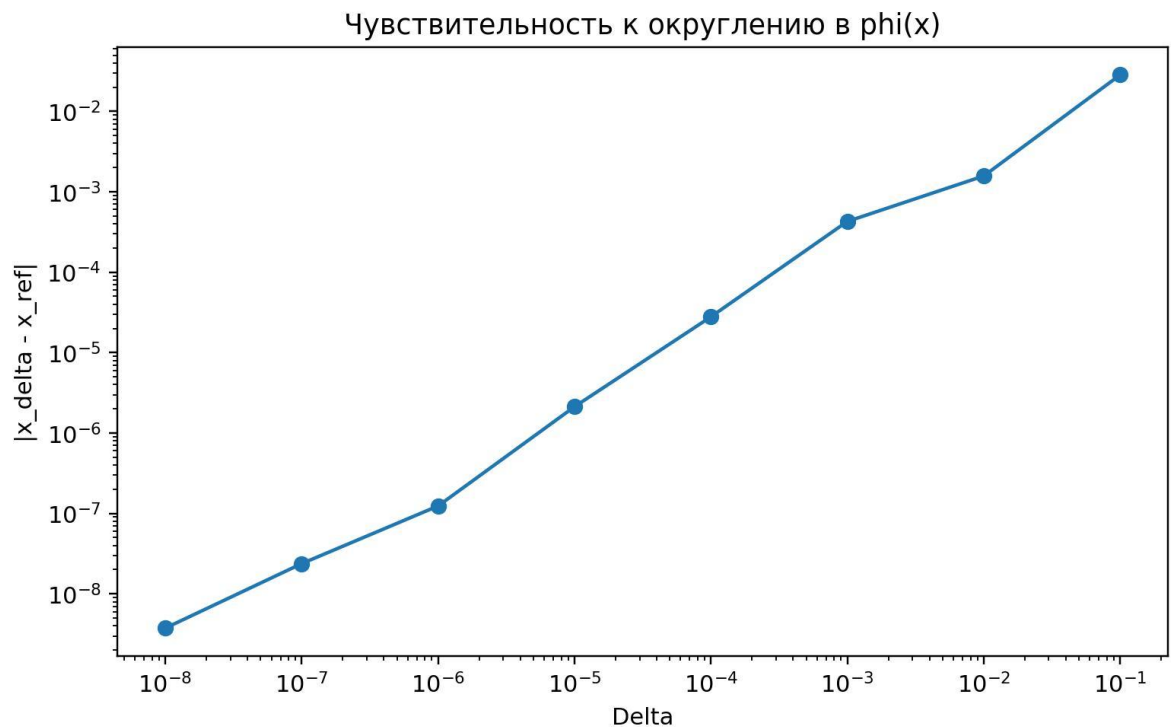


Рисунок 3 — Зависимость абсолютной ошибки $|\Delta x - x_{\text{ref}}|$ от параметра Δ .

С уменьшением Δ абсолютная ошибка в целом снижается. Уже при $\Delta \leq 10^{-5}$ найденное значение корня становится очень близким к эталонному, а при $\Delta = 10^{-7}$ и $\Delta = 10^{-8}$ отличие составляет всего несколько единиц последнего знака.

Вывод.

В ходе лабораторной работы для функции $f(x) = \sin(x^2) - 6x + 1$ был реализован метод простых итераций. Корень уравнения был изолирован на интервале $[0; 0.5]$, после чего исходное уравнение было преобразовано к виду $x = \varphi(x)$, где $\varphi(x) = (\sin(x^2) + 1)/6$.

Для выбранной итерационной функции на интервале $[0; 0.5]$ выполнено условие сходимости $|\varphi'(x)| \leq q \approx 0.161485 < 1$, поэтому метод сходится. Эталонное значение корня составило $x_{ref} \approx 0.171572123742972$.

Численные эксперименты показали, что метод обладает линейной скоростью сходимости: при увеличении требуемой точности число итераций растёт постепенно, а при $Eps = 10^{-6}$ требуется 7 шагов.

Исследование обусловленности показало, что при малых значениях Δ вычисленный корень практически совпадает с эталонным, тогда как грубое округление заметно искажает результат и может даже нарушить сходимость метода. Следовательно, для данной функции метод простых итераций применим и даёт устойчивые результаты при корректном выборе $\varphi(x)$ и достаточной точности вычислений.

ПРИЛОЖЕНИЕ А

Название файла: methods.h

```
#pragma once

#include <functional>

struct IterResult {
    double x;      // последнее приближение
    double lastDiff; //  $|x_{(n+1)} - x_n|$  на последнем шаге
    int iters;     // число выполненных итераций
    bool ok;       // true, если достигнут критерий остановки
};

// Округление значения до ближайшего числа, кратного delta.
// Если delta <= 0, значение возвращается без изменений.
double Round(double value, double delta);

// Оценка  $q = \max |\phi'(x)|$  на [left, right] равномерным перебором точек.
double EstimateQ(const std::function<double(double)>& dphi,
    double left,
    double right,
    int samples = 10000);

// Метод простых итераций для уравнения  $x = \phi(x)$ .
// phi - итерационная функция
// x0 - начальное приближение
// eps - требуемая точность по условию  $|x_{(n+1)} - x_n|$ 
// nmax - максимально допустимое число итераций
IterResult ITER(const std::function<double(double)>& phi,
    double x0,
    double eps,
    int nmax = 1000);
```

Название файла: methods.cpp

```
#include "methods.h"
```

```
#include <cmath>
```

```
// =====
```

```
// Округление значения до ближайшего числа, кратного delta
```

```
// =====
```

```
double Round(double value, double delta) {
```

```
    if (delta <= 0.0) {
```

```
        return value;
```

```
    }
```

```
    return std::round(value / delta) * delta;
```

```
}
```

```
// =====
```

```
// Оценка  $q = \max |\phi'(x)|$  на  $[left, right]$ 
```

```
// =====
```

```
double EstimateQ(const std::function<double(double)>& dphi,
```

```
                double left,
```

```
                double right,
```

```
                int samples) {
```

```
    if (samples < 1) {
```

```
        samples = 1;
```

```
    }
```

```
    double q = 0.0;
```

```
    for (int i = 0; i <= samples; ++i) {
```

```
        const double t = static_cast<double>(i) / samples;
```

```
        const double x = left + (right - left) * t;
```

```
        const double value = std::fabs(dphi(x));
```

```
        if (value > q) {
```

```
            q = value;
```

```
        }
```

```
    }
```

```
    return q;
```

```
}
```

```
// =====
```

```
// Метод простых итераций
```

```
// =====
IterResult ITER(const std::function<double(double)>& phi,
                double x0,
                double eps,
                int nmax) {
    IterResult result{};
    result.x = x0;
    result.lastDiff = 0.0;
    result.iters = 0;
    result.ok = false;

    double x = x0;
    for (int n = 1; n <= nmax; ++n) {
        const double y = phi(x);
        const double diff = std::fabs(y - x);

        result.x = y;
        result.lastDiff = diff;
        result.iters = n;

        if (diff < eps) {
            result.ok = true;
            return result;
        }

        x = y;
    }

    return result;
}
```

Название файла: main.cpp

```
#include "methods.h"

#include <cmath>
#include <fstream>
#include <iomanip>
#include <iostream>
```

```

#include <stdexcept>
#include <string>
#include <vector>

namespace {

constexpr double LEFT = 0.0;
constexpr double RIGHT = 0.5;
constexpr double X0 = 0.25;
constexpr int NMAX = 1000;

struct SpeedRow {
    double eps;
    int iters;
    double x;
    double absError;
    double residual;
};

struct ConditioningRow {
    double delta;
    int iters;
    double x;
    double absError;
    double residual;
    bool ok;
};

double f(double x) {
    return std::sin(x * x) - 6*x + 1;
}

double phi(double x) {
    return (std::sin(x*x)+1.0)/6.0;
}

double dphi(double x) {
    return (x*std::cos(x*x))/3.0;
}

double phiRounded(double x, double delta) {
    return Round(phi(x), delta);
}

void WriteSpeedCsv(const std::string& filename,
                  const std::vector<SpeedRow>& rows) {
    std::ofstream out(filename);
    out << "eps,iters,x,abs_error,residual\n";
    out << std::setprecision(17);
    for (const auto& row : rows) {
        out << row.eps << ','
            << row.iters << ','
            << row.x << ','
            << row.absError << ','
            << row.residual << "\n";
    }
}

```

```

    }
}

void WriteConditioningCsv(const std::string& filename,
                        const std::vector<ConditioningRow>& rows) {
    std::ofstream out(filename);
    out << "delta,status,itors,x,abs_error,residual\n";
    out << std::setprecision(17);
    for (const auto& row : rows) {
        out << row.delta << ','
            << (row.ok ? "ok" : "nmax") << ','
            << row.itors << ','
            << row.x << ','
            << row.absError << ','
            << row.residual << "\n";
    }
}

int main() {
    try {
        std::cout << std::fixed << std::setprecision(15);

        const double q = EstimateQ(dphi, LEFT, RIGHT);
        std::cout << "Function: f(x) = exp(1/x^2) - ln(x)\n";
        std::cout << "Interval: [" << LEFT << "; " << RIGHT << "]\n";
        std::cout << "Initial approximation x0 = " << X0 << "\n";
        std::cout << "q = max |phi'(x)| on [" << LEFT << "; " << RIGHT << "] = " << q <<
            "\n\n";

        const auto reference = ITER(phi, X0, 1e-14, NMAX);
        const double xref = reference.x;
        std::cout << "Reference run (delta = 0, eps = 1e-14):\n";
        std::cout << "  xref = " << xref << "\n";
        std::cout << "  iters = " << reference.itors << "\n";
        std::cout << "  |f(xref)| = " << std::fabs(f(xref)) << "\n\n";

        const std::vector<double> epsValues = {
            1e-1, 1e-2, 1e-3, 1e-4,
            1e-5, 1e-6, 1e-7, 1e-8
        };

        std::vector<SpeedRow> speedRows;
        std::cout << "Speed of convergence:\n";
        for (double eps : epsValues) {
            const auto result = ITER(phi, X0, eps, NMAX);
            const SpeedRow row{
                eps,
                result.itors,
                result.x,
                std::fabs(result.x - xref),
                std::fabs(f(result.x))
            };
            speedRows.push_back(row);
        }
    }
}

```



```

        std::cout << "  eps = " << std::scientific << eps
            << ", iters = " << std::fixed << row.iters
            << ", x = " << std::setprecision(15) << row.x
            << ", |x - xref| = " << std::scientific << row.absError
            << ", |f(x)| = " << row.residual << "\n";
    }
    std::cout << "\n";

    const std::vector<double> deltaValues = {
        1e-1, 1e-2, 1e-3, 1e-4,
        1e-5, 1e-6, 1e-7, 1e-8
    };

    std::vector<ConditioningRow> conditioningRows;
    std::cout << "Conditioning experiment (eps = 1e-8):\n";
    for (double delta : deltaValues) {
        const auto result = ITER(
            [delta](double x) {
                return phiRounded(x, delta);
            },
            X0,
            1e-8,
            NMAX
        );

        const ConditioningRow row{
            delta,
            result.iters,
            result.x,
            std::fabs(result.x - xref),
            std::fabs(f(result.x)),
            result.ok
        };
        conditioningRows.push_back(row);

        std::cout << "  delta = " << std::scientific << delta
            << ", status = " << (row.ok ? "ok" : "nmax")
            << ", iters = " << std::fixed << row.iters
            << ", x = " << std::setprecision(15) << row.x
            << ", |x - xref| = " << std::scientific << row.absError
            << ", |f(x)| = " << row.residual << "\n";
    }

    WriteSpeedCsv("speed_results.csv", speedRows);
    WriteConditioningCsv("conditioning_results.csv", conditioningRows);

    return 0;
} catch (const std::exception& ex) {
    std::cerr << "Error: " << ex.what() << "\n";
    return 1;
}
}

```

Название файла: plot.py

```
import csv
import math
from pathlib import Path
import matplotlib.pyplot as plt

def f(x: float) -> float:
    return math.exp(1.0 / (x * x)) - math.log(x)

def read_csv(path: str):
    with open(path, newline="", encoding='utf-8') as fobj:
        return list(csv.DictReader(fobj))

base = Path('.')
speed_rows = read_csv(base / 'speed_results.csv')
cond_rows = read_csv(base / 'conditioning_results.csv')

# число итераций в зависимости от требуемой точности
xs_speed = [-math.log10(float(row['eps'])) for row in speed_rows]
ys_speed = [int(row['iters']) for row in speed_rows]

plt.figure(figsize=(7.2, 4.6))
plt.plot(xs_speed, ys_speed, marker='o')
plt.xlabel('-log10(Eps)')
plt.ylabel('Число итераций')
plt.title('Метод простых итераций: скорость сходимости')
plt.tight_layout()
plt.savefig('figure_speed.png', dpi=220)
plt.close()

# абсолютная погрешность в зависимости от шага округления
xs_cond = [float(row['delta']) for row in cond_rows if row['status'] == 'ok']
ys_cond = [float(row['abs_error']) for row in cond_rows if row['status'] == 'ok']

plt.figure(figsize=(7.2, 4.6))
plt.loglog(xs_cond, ys_cond, marker='o')
plt.xlabel('Delta')
plt.ylabel('|x_delta - x_ref|')
plt.title('Чувствительность к округлению в phi(x)')
plt.tight_layout()
plt.savefig('figure_conditioning.png', dpi=220)
plt.close()
```

ПРИЛОЖЕНИЕ Б

Название файла: speed_results.csv

```
eps,iters,x,abs_error,residual
0.10000000000000001,1,0.17707655297373004,0.005504429230758473,
0.031108350225244052
0.01,2,0.17189182793618937,0.00031970419321780685,0.00180846604
37986955
0.001,3,0.17159041692888957,1.8293185918005772e-
05,0.00010348429959328165
```

```
0.0001,4,0.17157316954562404,1.0458026524773345e-
06,5.9161091319204218e-06
1.0000000000000001e-
05,5,0.17157218352743539,5.9784463823930878e-08,3.3820097300463203e-
07
9.999999999999995e-
07,5,0.17157218352743539,5.9784463823930878e-08,3.3820097300463203e-
07
9.999999999999995e-
08,6,0.17157212716060655,3.4176349805736805e-09,1.9333576961599874e-
08
1e-08,7,0.17157212393834373,1.953721628922267e-
10,1.1052219139884301e-09
```

Название файла: conditioning_results.csv

```
delta,status,itors,x,abs_error,residual
0.10000000000000001,ok,2,0.20000000000000001,0.0284278762570284
45,0.16001066581336598
0.01,ok,3,0.17000000000000001,0.001572123742971554,0.0088959772
39829489
0.001,ok,3,0.17200000000000001,0.00042787625702844778,0.0024203
151950012991
0.0001,ok,4,0.1716,2.7876257028436324e-
05,0.00015769533369058841
1.0000000000000001e-05,ok,5,0.17157,2.1237429715659228e-
06,1.2014027866880106e-05
9.999999999999995e-07,ok,6,0.171572,1.2374297156392267e-
07,7.0001455110269006e-07
9.999999999999995e-
08,ok,7,0.17157210000000001,2.3742971561047099e-
08,1.3431409473518841e-07
1e-08,ok,8,0.17157211999999999,3.7429715715742162e-
09,2.1174005881974267e-08
```